ITS Symposium manuscript No.
(will be inserted by the editor)

ReRAPTOR: Reverse-Objective RAPTOR for Time-Budgeted Value-Maximizing Transit Itinerary Planning

Bin Zhou · Guangxian Li · Teruki Ogo · Yutaka Arakawa · Tsunenori Mine

Received: date / Accepted: date

Abstract Conventional public-transit routing minimizes travel time, whereas leisure trips may seek the most valuable timetable-feasible itinerary within a fixed time budget. We propose ReRAPTOR, a RAPTOR-style label-setting method that ranks partial routes by a task-specific value surrogate. Its search is kept tractable by two-stage Beam control, heuristic Dominance reduction, and optional Negative-penalty down-ranking. On Fukuoka timetable data, a seeded stop-reward benchmark compares ReRAPTOR with nine routing and metaheuristic baselines. ReRAPTOR obtains the highest mean Stop Score (872.44), 8.71% above NSGA-II, and is 6.89× faster on average. The advantage comes mainly from finding feasible routes that visit more distinct scored stops; on queries where every NSGA-II run succeeds, the quality difference is not conclusive. Losses cluster at several long-budget origins. Ablation shows that Beam and Dominance provide the main scalability benefit. Across 22 sensitivity configurations, results are unchanged for $W \geq 25$ and for all tested tie-breaker coefficients, while candidate trimming controls the quality–resource trade-off. ReRAPTOR therefore offers a strong, but not universal, quality–speed balance for value-oriented transit itinerary planning.

Keywords Transit itinerary planning · Transit schedule data · RAPTOR · Time-budgeted route planning · Multi-objective search

Bin Zhou and Guangxian Li
Graduate School of Information Science and Electrical Engineering, Kyushu University, Fukuoka, Japan
E-mail: zhou.bin.883@s.kyushu-u.ac.jp; li.guangxian.021@s.kyushu-u.ac.jp

Teruki Ogo
Faculty of Information Science and Engineering, Kyushu Sangyo University, Fukuoka, Japan
E-mail: k25rm021@st.kyusan-u.ac.jp

Yutaka Arakawa and Tsunenori Mine
Faculty of Information Science and Electrical Engineering, Kyushu University, Fukuoka, Japan
E-mail: arakawa@ait.kyushu-u.ac.jp; mine@ait.kyushu-u.ac.jp

1 Introduction

Public-transit routing usually seeks earliest-arrival or Pareto-efficient journeys [2,4,6,12]. For leisure travel, however, a user may instead want the most rewarding itinerary that can be completed within a fixed time. Route value may depend on the stops visited, their rewards, spatial coverage, and route history. The problem is therefore closer to budgeted itinerary design than to shortest-path search [11,21,22].

Combining timetable feasibility with a history-dependent objective is difficult. Every extension must respect scheduled departures, arrivals, and transfers, but many partial routes can reach the same stop with different future value. Keeping only an earliest-arrival label loses useful history; keeping all labels makes the search grow rapidly.

We address this gap with ReRAPTOR, a reverse-objective RAPTOR-style search over timetable schedules [6,24,25]. It retains timetable-feasible expansion but ranks partial itineraries with a pluggable value module. Two-stage Beam control limits expansion width, heuristic Dominance removes locally weaker labels, and an optional Negative penalty down-ranks semantically poor routes.

Experiments on Fukuoka schedules compare ReRAPTOR with nine baselines under a reproducible stop-reward task. We report overall quality and time, explain the meaning of a Stop Score gain, identify budgets

and origins where the method loses, isolate its search controls, test 22 parameter configurations, and examine three route cases. The contribution is thus both methodological—a value-aware timetable search with pluggable objectives—and empirical: evidence about its quality–speed balance, the mechanisms behind it, and its limits.

2 Related Work

RAPTOR is a round-based framework for earliest-arrival and multicriteria transit routing [6]. Later work improves transfers, restricted Pareto search, and acceleration [1,2,4,5,12,19]. Fare-aware and broader multi-objective variants have also been studied [8,23], supported by GTFS and related timetable datasets [10,14, 17,25]. These methods make timetable routing efficient, but do not directly optimize history-dependent route value under a fixed total budget.

Tourist trip design and orienteering instead maximize collected value under time constraints [9,11,21, 22]. Preference-aware route systems extend this idea to personalized and multimodal settings [18,20,27,26]. General optimizers such as NSGA-II, ILS, VNS, and SA are also relevant [3,13,15,16]. ReRAPTOR differs by optimizing value during timetable-feasible expansion, not only after full itineraries have been constructed.

3 Problem Formulation

Let $G = (\mathcal{S}, \mathcal{E})$ be a timetable graph, where an edge $e = (u, v, \tau^{\mathrm{dep}}, \tau^{\mathrm{arr}}, \ell)$ is a scheduled ride from stop u to v on line ℓ. For a query $q = (o, t_0, B)$, we seek a route $r = (e_1, \ldots, e_m)$ that starts at o, departs no earlier than t_0, respects stop continuity and a minimum transfer buffer Δ_{tr}, and finishes within budget B:

$$\begin{aligned} u(e_1) = o, \qquad & \tau^{\mathrm{dep}}(e_1) \geq t_0, \\ v(e_i) = u(e_{i+1}), & \\ \tau^{\mathrm{dep}}(e_{i+1}) \geq \tau^{\mathrm{arr}}(e_i) + \delta(e_i, e_{i+1}),\ & \tau^{\mathrm{arr}}(e_m) \leq t_0 + B. \end{aligned} \tag{1}$$

Here $\delta = 0$ for continuation on the same trip and $\delta = \Delta_{\mathrm{tr}}$ otherwise. A pluggable module supplies the complete route value $F_\theta(r)$, the partial-route ranking score $S_\theta(\lambda)$, and any additional acceptance rule.

Experiments 1–3 use a seeded stop-reward module. Each stop receives a fixed reward $w_\omega(s)$; every unique non-origin stop also incurs dwell time η. If $V^+(r)$ is the set of such stops and $T(r)$ is elapsed time,

$$\begin{aligned} T_\eta(r) &= T(r) + \eta |V^+(r)|, \qquad T_\eta(r) \leq B, \\ F_{\omega,\eta}(r) &= \sum_{s \in V^+(r)} w_\omega(s). \end{aligned} \tag{2}$$

The reward can represent an attraction rating or another utility, while the same timetable search can use a different value module.

4 Proposed Method

4.1 Search State and Expansion

ReRAPTOR is a label-setting search over timetable-feasible stop schedules [6,12]. A label records arrival time, ride time, distance, visited stops, line and stop counts, transfers, the ordered leg sequence, and objective-specific state. For the stop-reward task, this state includes collected reward σ and the number of unique scored stops. Extending a label along a feasible scheduled edge updates these fields and adds a stop reward only on its first scored visit. Unlike an earliest-arrival label, this richer state preserves information needed to estimate the future value of a partial itinerary.

4.2 Value Ranking and Search Control

The objective module ranks partial labels by

$$S_\theta(\lambda) = \Phi_\theta(\lambda) - P(\lambda), \tag{3}$$

where Φ_θ is task utility and P only lowers rank; it does not reject a label. The default penalty is 50,000 after a stop is visited more than twice and 20,000 when ride minutes per visited stop fall below two. These conservative defaults suppress clear loops and very short-hop chains. In the reported stop-reward experiments,

$$\begin{aligned} S_{\omega,\eta} = 10000\sigma + 5\,\mathrm{SPH} - 0.1T_\eta \\ - 50000\max(0, T_\eta - B) - P, \end{aligned} \tag{4}$$

where $\mathrm{SPH} = 60\sigma/T_\eta$. Reward is deliberately dominant; SPH and time break ties. The coefficients were fixed before evaluation and are varied in Experiment 3.

Three controls limit growth. First, *candidate trimming* ranks outgoing edges by same-trip continuity, objective priority, and schedule time, then keeps at most $C(B) = 30$ edges for $B \leq 240$ min and 20 otherwise. Second, *heuristic Dominance* compares labels in the same stop–round bucket:

$$\lambda^a \succeq \lambda^b \iff \sigma(\lambda^a) \geq \sigma(\lambda^b) \ \wedge \ T_\eta(\lambda^a) \leq T_\eta(\lambda^b), \tag{5}$$

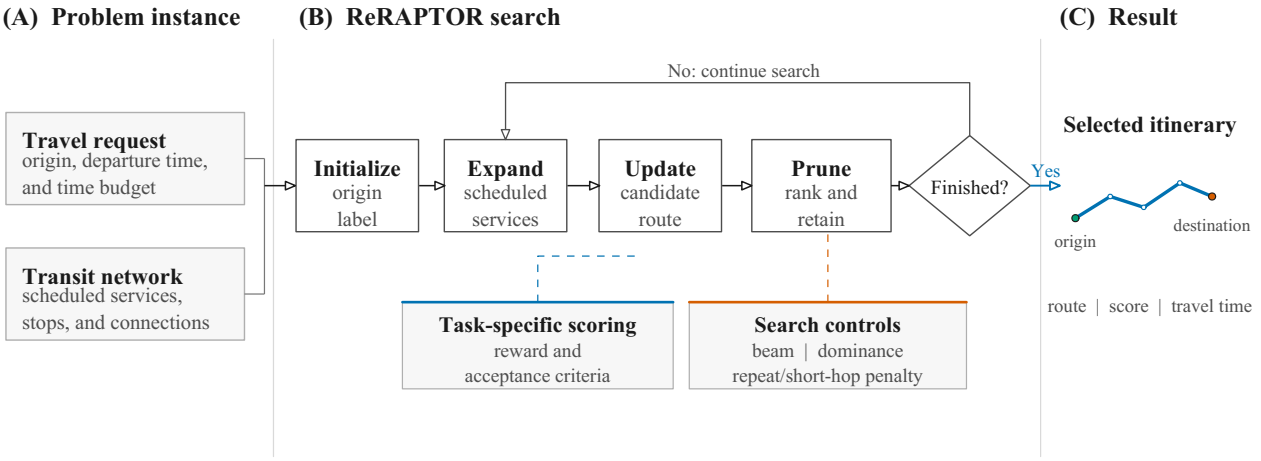


Fig. 1 ReRAPTOR workflow. A common timetable-feasible search backbone uses a task-specific scoring and acceptance module; Experiment 2 disables the three search controls individually and in combination.

with one strict inequality or a higher surrogate score for an exact tie. This rule is not Pareto-safe because visited sets and continuation states can differ; it is a practical label reduction [7,12]. Third, bucket-level *Beam control* keeps the best $W = 50$ labels by S_θ. The default is a conservative middle width selected before sensitivity testing, not an empirically optimized value. `NoBeam` disables both candidate trimming and bucket truncation. The final answer is the highest-scored feasible label with at least two legs.

4.3 Complexity and Implementation

Each label expands at most $C(B)$ edges and each stop–round bucket stores at most W labels, giving $O(|\mathcal{S}|W)$ memory per round. The implementation expands stop schedules directly, memoizes geometry by label signature, uses a 120-min lookahead, and limits rounds to $R = \min(220, \max(40, \lfloor B/2 \rfloor + 20))$. Timeouts and label caps prevent unbounded runs and are counted as resource-limit hits in the experiments.

5 Experiments

5.1 Experiment 1: Comprehensive Algorithm Comparison

5.1.1 Setup

Experiment 1 tests quality, runtime, and robustness on the stop-reward task. Baseline selection follows three criteria: (1) task compatibility with timetable-feasible fixed-budget routing, (2) reproducible implementation in our evaluation pipeline, and (3) diversity across optimizer families. We therefore compare general-purpose metaheuristics (NSGA-II [3], ILS+2opt [15], VNS [16], SA [13]), transit-routing baselines (McRAPTOR, Pure RAPTOR [6], Greedy), and two recent adapted methods described below.

Adapted baselines. ALNS and DynPref address related but different systems, so Table 1 states exactly what was retained and changed. Both use the same timetable repair, validator, reward map, and budget rule as all other methods; they are algorithmic adaptations, not replications of the original application settings.

The data contain 304,344 timetable segments, 1,032 stops, and 302 lines. The grid combines 10 origins, three departures (07:00, 10:00, 14:00), four budgets (2h, 4h, 6h, 8h), and five reward seeds, giving 600 matched tasks per method. After sorting station codes, each seed $\omega \in \{20260323, \ldots, 20260327\}$ draws $w_\omega(s) \sim \mathcal{U}\{1, \ldots, 100\}$ independently once per stop. The map is then fixed across methods. A unique non-origin stop is scored once and adds 10 min dwell; the origin adds neither reward nor dwell. Elapsed time includes initial waiting.

Stochastic methods use algorithm seeds $0, \ldots, 4$, averaged within each matched task; deterministic methods run once. Invalid or empty routes remain as zero-score failures. All methods share inputs and validation but retain their own search guards. Runs were sequential on Windows 11 (build 26200), Intel i5-12600KF (10 physical/16 logical cores), 31.84 GiB RAM, Python 3.12.7, NumPy 1.26.4, pandas 2.2.2, SciPy 1.13.1, and NetworkX 3.3. Wall time excludes loading and post-validation; no query-level parallelism

Table 1 Implementation of the two adapted baselines (n: route length; u: effective-budget use). Complete parameter files accompany the code.

Method	Retained idea	Fixed-timetable adaptation	Search budget
ALNS (adapted)	Adaptive destroy–repair, operator weights, and SA acceptance [28].	Four tail cuts (shallow, medium, deep, restart) are repaired through the shared timetable interface for at most 44 steps. Exploration rates are 0.20/0.32/0.48/0.55; weights update every 12 iterations (reaction 0.30, rewards 5/2/0.5); $T_0 = 6$, cooling 0.985, floor 0.6.	160 iterations or 45 s; seeds $0, \ldots, 4$.
DynPref (adapted)	Rolling shift from exploration to efficiency [27].	Shared-mobility state is removed. Early ($u < .35$), middle ($.35 \leq u < .70$), and late stages use progressively shallower tail cuts, exploration rates 0.50/0.38/0.22, and a score that shifts from distinct-stop coverage to efficiency while reward remains primary.	24 failed repairs or 45 s; seeds $0, \ldots, 4$.

Table 2 Overall summary of Experiment 1 (query-level means, 120 queries).

Method	Stop Score	Elapsed (ms)	Score / hour	Avg. rank
Ours (Full)	872.44	5171.18	185.47	1.78
NSGA-II	802.54	35634.78	164.94	3.20
ILS+2opt	758.73	638.40	162.30	2.87
ALNS (adapted)	709.83	104.87	153.52	4.44
VNS	700.27	224.57	151.64	4.88
SA	622.12	168.31	134.63	6.48
McRAPTOR	539.76	2994.83	146.18	6.85
Greedy (L=3)	459.36	3.67	103.51	8.40
DynPref (adapted)	459.10	13.39	107.43	8.07
Pure RAPTOR	316.73	56470.87	163.96	8.04

was used. Paired 95% intervals use 20,000 crossed block-bootstrap replications over reward seeds and origin–departure blocks.

5.1.2 Main Results

Table 2 gives the overall (2h+4h+6h+8h) query-level means. ReRAPTOR achieves the highest average stop score (872.44), exceeding NSGA-II (802.54) by +8.71%, while also being 6.89× faster overall (5171.18 ms vs 35634.78 ms). Together, these results show that ReRAPTOR combines a high average score with a shorter generation time across many queries.

In Table 2, *Stop Score* is the query-level mean of `stop_score_total` (i.e., $F_{\omega,\eta}(r)$ in Eq. 2); *Elapsed (ms)* is query-level mean wall-clock generation time; *Score / hour* is SPH $= F_{\omega,\eta}(r)/(T_\eta(r)/60)$; and *Avg. rank* orders methods by higher score then lower elapsed time.

5.1.3 Interpreting the Stop-Score Improvement

The absolute difference of 69.899 points is 8.710% of the NSGA-II mean. Because the benchmark draws each stop reward from $\mathcal{U}\{1, \ldots, 100\}$, whose expectation is 50.5, this gap is numerically equivalent to 1.384 expected-reward stops (95% crossed block-bootstrap CI $[0.159, 2.592]$). This conversion is only an experimental-unit interpretation: it is not a count of real attractions, monetary value, or user satisfaction.

The directly observed task metrics support a coverage-based interpretation. ReRAPTOR visits 1.320 more unique scored stops on average (95% CI $[0.535, 2.092]$), while its selected reward per scored stop is 1.914 points lower on the 581 tasks where NSGA-II returns at least one feasible route (95% CI $[-3.247, -0.654]$). Thus, the aggregate advantage is associated primarily with collecting value from more distinct stops and with returning feasible routes reliably, rather than selecting stops with higher mean reward.

At the task level, ReRAPTOR beats NSGA-II in 380/600 comparisons, ties in 49, and loses in 171. After averaging over stop-score seeds, ReRAPTOR achieves a higher stop score and shorter runtime than NSGA-II in 92/120 base queries. Figure 2 visualizes this combined view and summarizes the main result of Experiment 1.

5.1.4 Budget and Seed Robustness

ReRAPTOR is higher-scoring and faster than NSGA-II at every budget. Their mean scores are 398.61/327.98, 803.75/656.50, 988.91/965.94, and 1298.49/1259.73 for 2h–8h; corresponding ReRAPTOR runtimes are 82.66, 2042.99, 3166.40, and 15392.68 ms, versus 23221.84, 33730.70, 40131.44, and 45455.16 ms. The score gap is positive for all five reward seeds (+41.55 to +117.62). All 600 ReRAPTOR tasks return a positive feasible route.

5.1.5 Reliability and Conditions of Underperformance

The mean paired gap is +69.899 points (95% crossed block-bootstrap CI $[+8.054, +130.917]$), but its distribution is heterogeneous. The mean gaps for 2h, 4h, 6h, and 8h are +70.633, +147.243, +22.967, and +38.753, respectively; only the 2h and 4h intervals exclude zero. Across

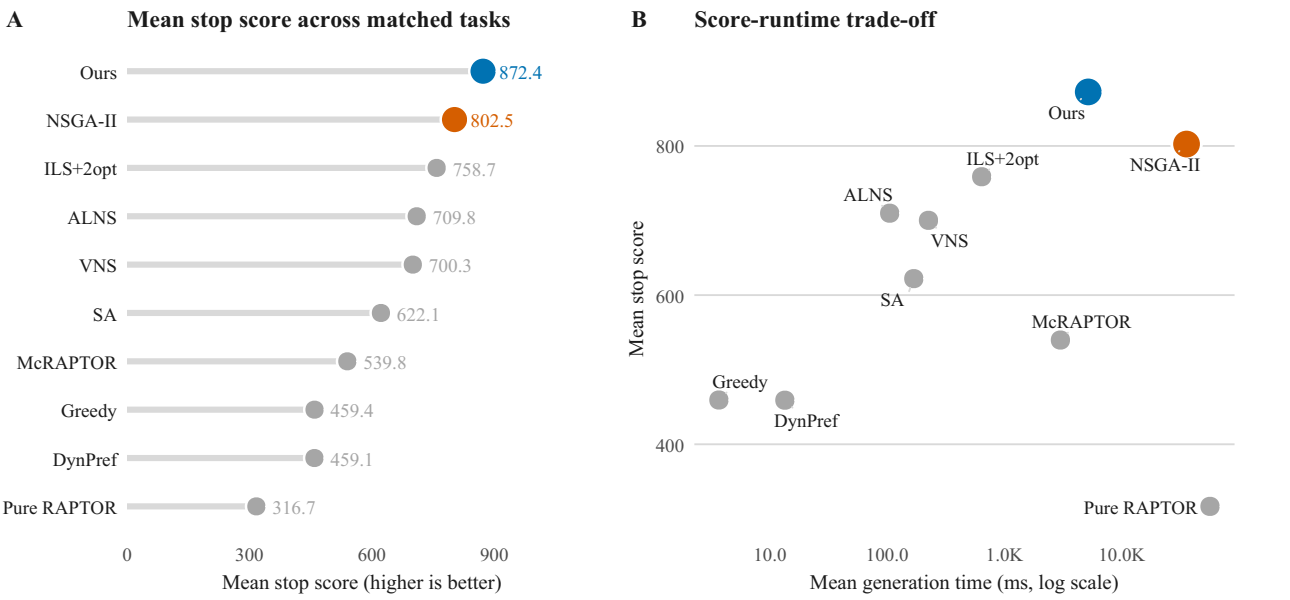


Fig. 2 Overview of Experiment 1 under the score-guided multi-seed benchmark. Panel A ranks methods by mean stop score, and Panel B shows the quality–generation-time landscape on a logarithmic time axis.

all losses, the conditional mean loss magnitude is 236.423 points, and the mean gap in the worst 10% of tasks is −556.350. The differences are clearer across origins than across departure times. Noke is the clearest underperformance location, with a mean gap of −225.693 (95% CI [−361.123, −88.403]) and 33/60 losses. Its 6h and 8h gaps are −468.920 and −687.200. Ohori-koen and Yakuin also turn negative at 6h and 8h, although their origin-level intervals cross zero. By departure, the 10:00 block is positive (+120.373, 95% CI [14.141, 207.916]); the 07:00 and 14:00 intervals cross zero. Figure 3 shows that the largest losses cluster in particular long-budget origin cells rather than occurring uniformly. This breakdown shows where the losses occur, but it does not prove that network topology caused them.

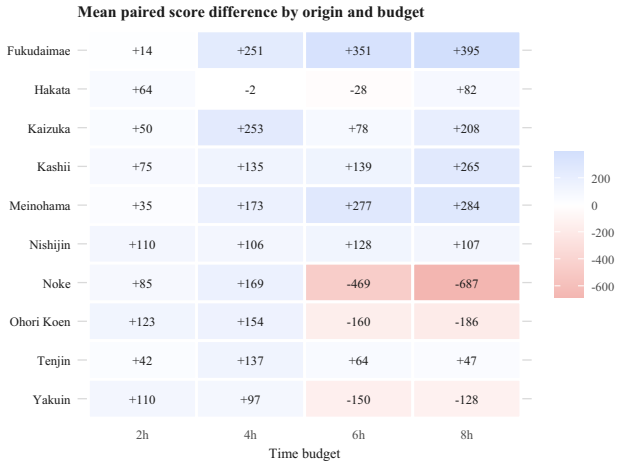


Fig. 3 Mean paired Stop Score gap (ReRAPTOR minus NSGA-II) by origin and budget. Each cell averages 15 matched tasks (three departures and five reward seeds); blue favors ReRAPTOR and red favors NSGA-II. Algorithm failures remain zero-score outcomes under the primary all-task policy.

The primary analysis treats reliability as part of overall algorithm performance. NSGA-II has 288/3000 algorithm-seed failures, affecting 148/600 paired tasks. When analysis is restricted to the 452 tasks on which all five NSGA-II runs are feasible, the mean gap is +4.708 (95% CI [−63.024, 67.875]); averaging only feasible NSGA-II seeds gives −3.059 over 581 tasks (95% CI [−71.301, 62.440]). The headline result therefore measures overall algorithm performance, including the ability to return a feasible itinerary. Once the comparison is limited to successful runs, the data do not show a clear difference in solution quality.

5.2 Experiment 2: Three-Component Search-Control Ablation and Supporting Evidence

5.2.1 Protocol and Results

Experiment 2 disables Beam, Dominance, and Negative down-ranking individually and in combination on the same 600 tasks. Eight deterministic configurations produce 4,800 runs. We report score, time, explored labels,

Table 3 Canonical Experiment 2 ablation (means over 600 matched task units per configuration).

Method	StopScore	Elapsed (ms)	Labels	CapHit (%)
Ours (Full)	872.44	10288.03	91.4K	4.67
Ours (NoNeg)	872.47	11433.06	93.6K	5.50
Ours (NoDom)	787.46	24141.25	219.7K	16.00
Ours (NoBeam)	705.72	43041.76	539.5K	67.67
Ours (NoBeam-NoDom)	299.87	56597.52	882.5K	91.67
Ours (NoDom-NoNeg)	798.04	20688.47	231.9K	11.67
Ours (NoBeam-NoNeg)	790.94	42042.27	1.001M	65.50
Ours (NoBeam-NoDom-NoNeg)	300.59	41234.67	903.9K	91.17

and the fraction of tasks stopped by the 60-s timeout or no-pruning label cap.

Table 3 shows why the combination works. Removing Dominance lowers score by 84.98, multiplies time by 2.35, and labels by 2.40. Removing both width controls lowers score by 166.72, multiplies time by 4.18 and labels by 5.90, and raises resource-limit incidence from 4.67% to 67.67%. Without both Beam and Dominance, 91.67% of tasks hit a limit and score falls by 572.57: the larger state space prevents productive deep search from finishing. The effect strengthens with budget. `NoBeam` is slightly better at 2h (411.79 vs 398.61) but already 8.71× slower; at 6h and 8h every task times out.

The components therefore have different roles. Beam and Dominance are complementary scalability controls. Negative down-ranking is objective-specific: `NoNeg` ties Full on 594/600 tasks and improves six by only 0.03 points on average, so this benchmark does not establish a generic gain from it.

5.3 Experiment 3: Balance and Hyperparameter Sensitivity

5.3.1 Balance and Protocol

On the 120 base queries, ReRAPTOR is closest to the normalized score–time ideal point (distance 0.1315, versus 0.7413 for NSGA-II) and is both higher-scoring and faster in 92/120 queries. This motivates testing whether the result depends on $W = 50$, the candidate cap, or surrogate coefficients.

5.3.2 Sensitivity Protocol

The one-at-a-time panel uses the same 600 tasks per configuration. It tests $W = \{10, 25, 50, 100, 200\}$; short/long-budget caps $15/10, 30/20, 60/40$, or none; reward weights $5000, 10000, 20000$; SPH weights $0, 2.5, 5, 10$; effective-time weights $0, .05, .1, .2$; and Negative-penalty scales $.5, 1, 2$.

OAT score differences are paired on the exact origin–departure–budget–reward-seed key. The reported 95% intervals use 20,000 crossed block-bootstrap replications over reward seeds and origin–departure scenarios, retaining budgets as repeated design points. A confirmatory 3×3 panel crosses $W \in \{25, 50, 100\}$ with caps 15/10, 30/20, and 60/40 on a 72-task subset. All runs are sequential on the same workstation; paired score and search-resource measures are the primary evidence.

5.3.3 Sensitivity Results and Interpretation

Figure 4 shows that the results are not sensitive to bucket width over most of the tested range. The default mean Stop Score is 872.438. $W = 25$, 100, and 200 reproduce that score on all 600 matched tasks. $W = 10$ differs on only eight tasks and changes the mean by -0.313 (95% CI $[-0.827, 0]$). Resource-cap incidence is 0.0%, 2.0%, 3.5%, 6.2%, and 7.0% for $W = 10, 25, 50, 100, 200$, respectively. Thus, $W = 50$ is a conservative middle-of-range default: it avoids the rare losses at $W = 10$, while widths above 50 add no score on this grid. $W = 25$ is also a viable lower-resource setting rather than evidence for a unique optimum at 50.

Candidate trimming controls the main trade-off. Tight caps 15/10 reduce mean score to 674.56, a paired change of -197.88 (95% CI $[-232.49, -164.07]$), but eliminate cap hits. Caps 60/40 increase mean score to 984.87, a change of $+112.43$ (95% CI $[+62.18, +171.16]$), at the cost of 362,108 mean labels and 48.5% cap incidence. Removing the cap also performs poorly: mean score falls to 706.69, mean labels rise to 542,458, and 67.7% of tasks exhaust a resource limit. The default 30/20 therefore occupies the stable middle of the quality–completion trade-off. The interaction subset supports the same mechanism: score is width-invariant at caps 15/10 and 30/20, whereas cap 60/40 adds 174–206 points but incurs 27.8%–40.3% cap incidence; increasing W does not give a consistent additional gain.

The surrogate coefficients have little effect. Halving or doubling the reward weight changes the mean by only -0.050 or $+0.027$; all score-per-hour and effective-time variants reproduce the default score on all 600 tasks; and Negative-penalty scales 0.5 and 2 change the mean by $+0.027$ and -0.063, respectively. These small effects match the designed priority hierarchy: collected reward dominates, while score-per-hour and effective time break ties. Overall, the conclusion is robust to the tested beam widths and coefficient ranges; the candidate cap is the parameter that materially controls the quality–resource boundary.

5.4 Experiment 4: Route Case Study

Three cases selected by fixed rules from the frozen 120-query results explain, rather than extend, the statistical

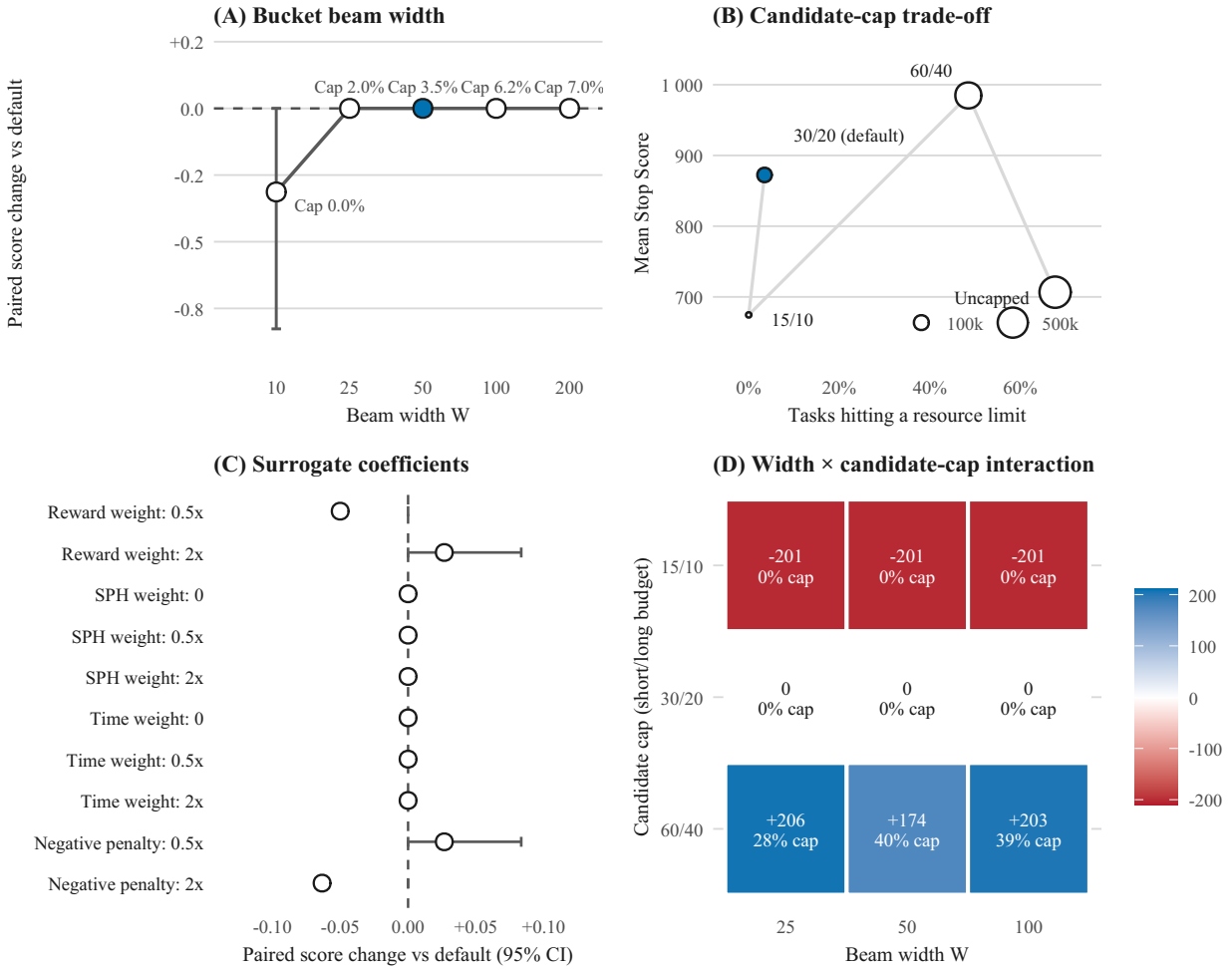


Fig. 4 Canonical hyperparameter sensitivity. (A) Paired score change and 95% crossed block-bootstrap interval across bucket beam widths; labels give resource-cap incidence. (B) Candidate-cap quality–resource trade-off, with point area proportional to explored labels. (C) Paired coefficient effects. (D) Confirmatory width–candidate-cap interaction; cells report mean score change from the $W = 50$, cap-30/20 subset default and cap incidence.

evidence. Case A is the largest score lead where ReRAPTOR is also faster. Case B is the smallest deficit where it ranks first by ideal distance but not score. Case C is the largest deficit among faster positive-score queries with a ReRAPTOR/NSGA-II ratio of at least 0.8.

Table 4 links the aggregate results to route behavior. Case A's gain comes mainly from scoring 21 stops rather than 11. In Case B, ReRAPTOR keeps 99.73% of Pure RAPTOR's score while running 94.3× faster; Greedy is faster still (0.85 ms) but scores only 263. In Case C, NSGA-II visits one fewer scored stop but earns 84.0 points per stop versus 70.4; candidate trimming can discard a partial route that later reaches a valuable long-budget sequence. Selection details and route maps are provided in the supplementary casebook.

6 Discussion

The experimental results support a more precise claim than simply "ReRAPTOR obtains the best routes." Under the seeded stop-reward benchmark, its key strength is the consistency of the quality–speed balance. In Experiment 1, ReRAPTOR achieves the highest average stop score, improves on NSGA-II by +8.71%, and remains 6.89× faster on average. Experiment 3 is consistent: ReRAPTOR has the best query-level ideal distance (0.1315) and is simultaneously higher-score and faster than NSGA-II in 92/120 base queries. Its sensitivity panel further shows that the score is unchanged for all tested $W \geq 25$ and for all score-per-hour and effective-time coefficient variants. Candidate trimming, rather than the exact bucket width or tie-breaker coefficient, determines the main quality–resource trade-off: overly

Table 4 Route cases. "Stops" counts unique scored non-origin stops.

Case	Query	ReRAPTOR			Comparator			Explanation
		Score	Time	Stops	Method	Score	Time	
A	Fukudaimae, 07:00, 8h	1455	13.83 s	21	NSGA-II	684	31.75 s	broader coverage
B	Kashii, 10:00, 2h	375	115.8 ms	6	Pure RAPTOR	376	10.92 s	near both ideals
C	Ohori-koen, 14:00, 8h	1479	4.11 s	21	NSGA-II	1680	50.92 s	lower reward density

tight caps discard productive extensions, while uncapped expansion frequently exhausts the search budget before deeper routes are completed. Experiment 4 also gives a more complete picture by showing that ReRAPTOR does not win every individual query.

The ablation results also clarify how the proposed method should be understood. Beam and Dominance are the two core scalability controls for keeping label search tractable. Removing Dominance sharply increases runtime and label count; removing Beam is even more expensive, especially at the 8-hour budget where cap hits occur in all tasks. Removing both leads to rapid search-space growth and lower route quality. Negative down-ranking has a different role: it is not the main speedup factor on the generic stop-reward benchmark. It remains an optional objective-alignment mechanism whose benefit must be tested under an explicitly instantiated penalty objective; the present random-reward results do not establish a generic quality gain. The seeded rewards are a controlled search benchmark rather than an estimate of tourist satisfaction. In an application, the reward map can be replaced by attraction ratings, user preferences, event value, or business priorities without changing timetable-feasible expansion. However, independent stationary rewards do not represent spatially correlated attractions, opening hours, arrival-time-dependent value, heterogeneous users, crowding, category diversity, or non-additive combinations of stops. The present evidence therefore supports objective-aligned search on reproducible value fields, not a claim about realized visitor utility.

Some findings are structural: history-dependent rewards make a single earliest-arrival label insufficient; bucket width control and heuristic dominance remove different forms of search growth; and a pluggable reward module can be attached to another timetable network. The numerical effect sizes, the origins and budgets where a method loses, and the best beam width may instead depend on Fukuoka's service frequency, transfer structure, centrality, and topology. We therefore expect the search design to transfer, but do not claim a universal performance advantage. External validation should cover cities with different network sizes, topologies, and service frequencies, together with learned or observed utility models.

7 Conclusion

This paper presented ReRAPTOR, a reverse-objective RAPTOR-style method for time-budgeted value-maximizing transit itinerary planning on timetable-based transit schedules. Its main methodological contribution is to optimize route value during timetable-feasible expansion through a shared search backbone with pluggable objective modules and a three-component search-control design. Across the main stop-reward benchmark, ReRAPTOR achieves the strongest overall quality–speed profile among the compared baselines. It attains the highest average stop score (872.44), exceeds NSGA-II by 8.71% on that metric, and remains 6.89× faster on average. Experiment 3 shows that it is also the closest method to the score–time ideal point and that its score is robust for $W \geq 25$ and across the tested surrogate coefficient ranges. The ablation and sensitivity studies jointly show that Beam and Dominance are the two core scalability components, whereas Negative down-ranking is an optional objective-alignment mechanism rather than a demonstrated generic speed or quality gain. The route case study illustrates both the strengths of the method and the cases in which it does not achieve the highest score.

Overall, ReRAPTOR is a practical approach for value-oriented transit itinerary planning when the goal is high route value with stable response time.

Acknowledgment

This study used transit schedule data for Fukuoka provided through Ekispert services (Powered by Ekispert API).

Supplementary Materials

Additional tables, source data files, and supplementary images will be released at `https://github.com/Akihito-zhou/reraptor_paper`.

References

1. Abuaisha, A., Wallace, M., Harabor, D., Shen, B.: Efficient and exact public transport routing via a transfer connection database. In: Proceedings of the International Symposium on Combinatorial Search, vol. 17, pp. 2–10. Association for the Advancement of Artificial Intelligence (AAAI) (2024). DOI 10.1609/socs.v17i1.31536
2. Agarwal, P., Rambha, T.: Scalable algorithms for bicriterion trip-based transit routing. IEEE Transactions on Intelligent Transportation Systems **25**(10), 14313–14327 (2024). DOI 10.1109/tits.2024.3391343
3. Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation **6**(2), 182–197 (2002). DOI 10.1109/4235.996017
4. Delling, D., Dibbelt, J., Pajor, T.: Fast and exact public transit routing with restricted pareto sets. In: 2019 Proceedings of the Twenty-First Workshop on Algorithm Engineering and Experiments (ALENEX), pp. 54–65. Society for Industrial and Applied Mathematics (2019). DOI 10.1137/1.9781611975499.5
5. Delling, D., Dibbelt, J., Pajor, T., Zündorf, T.: Faster transit routing by hyper partitioning. In: 17th Workshop on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2017), *Open Access Series in Informatics (OASIcs)*, vol. 59, pp. 8:1–8:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2017). DOI 10.4230/OASIcs.ATMOS.2017.8
6. Delling, D., Pajor, T., Werneck, R.F.: Round-based public transit routing. Transportation Science **49**(3), 591–604 (2015). DOI 10.1287/trsc.2014.0534
7. Demeyer, S., Goedgebeur, J., Audenaert, P., Pickavet, M., Demeester, P.: Speeding up martins' algorithm for multiple objective shortest path problems. 4OR **11**(4), 323–348 (2013). DOI 10.1007/s10288-013-0232-5
8. Euler, R., Lindner, N., Borndörfer, R.: Price optimal routing in public transportation. EURO Journal on Transportation and Logistics **13**, 100128 (2024). DOI 10.1016/j.ejtl.2024.100128
9. Gavalas, D., Konstantopoulos, C., Mastakas, K., Pantziou, G., Vathis, N.: Heuristics for the time dependent team orienteering problem: Application to tourist route planning. Computers & Operations Research **62**, 36–50 (2015). DOI 10.1016/j.cor.2015.03.016
10. Godfrid, J., Radnic, P., Vaisman, A., Zimányi, E.: Analyzing public transport in the city of buenos aires with mobilitydb. Public Transport **14**(2), 287–321 (2022). DOI 10.1007/s12469-022-00290-8
11. Gunawan, A., Lau, H.C., Vansteenwegen, P.: Orienteering problem: A survey of recent variants, solution approaches and applications. European Journal of Operational Research **255**(2), 315–332 (2016). DOI 10.1016/j.ejor.2016.04.059
12. He, P., Jiang, G., Lam, S.K., Sun, Y., Ning, F.: Exploring public transport transfer opportunities for pareto search of multicriteria journeys. IEEE Transactions on Intelligent Transportation Systems **23**(12), 22895–22908 (2022). DOI 10.1109/tits.2022.3194523
13. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. Science **220**(4598), 671–680 (1983). DOI 10.1126/science.220.4598.671
14. Kujala, R., Weckström, C., Darst, R.K., Mladenović, M.N., Saramäki, J.: A collection of public transport network data sets for 25 cities. Scientific Data **5**(1), 180089 (2018). DOI 10.1038/sdata.2018.89
15. Lourenço, H.R., Martin, O.C., Stützle, T.: Iterated local search. In: F. Glover, G.A. Kochenberger (eds.) Handbook of Metaheuristics, *International Series in Operations Research & Management Science*, vol. 57, pp. 320–353. Kluwer Academic Publishers (2003). DOI 10.1007/0-306-48056-5_11
16. Mladenović, N., Hansen, P.: Variable neighborhood search. Computers & Operations Research **24**(11), 1097–1100 (1997). DOI 10.1016/S0305-0548(97)00031-2
17. Pereira, R.H.M., Andrade, P.R., Vieira, J.P.B.: Exploring the time geography of public transport networks with the gtfs2gps package. Journal of Geographical Systems **25**(3), 453–466 (2023). DOI 10.1007/s10109-022-00400-x
18. Persia, F., Pilato, G., Ge, M., Bolzoni, P., D'Auria, D., Helmer, S.: Improving orienteering-based tourist trip planning with social sensing. Future Generation Computer Systems **110**, 931–945 (2020). DOI 10.1016/j.future.2019.10.028
19. Phan, D.M., Viennot, L.: Fast public transit routing with unrestricted walking through hub labeling. In: Analysis of Experimental Algorithms, pp. 237–247. Springer International Publishing (2019). DOI 10.1007/978-3-030-34029-2_16
20. Pilato, G., Persia, F., Ge, M., Chondrogiannis, T., D'Auria, D.: A modular social sensing system for personalized orienteering in the covid-19 era. ACM Transactions on Management Information Systems **14**(4), 1–26 (2023). DOI 10.1145/3615359
21. Ruiz-Meza, J., Montoya-Torres, J.R.: A systematic literature review for the tourist trip design problem: Extensions, solution techniques and future research lines. Operations Research Perspectives **9**, 100228 (2022). DOI 10.1016/j.orp.2022.100228
22. Souffriau, W., Vansteenwegen, P., Vertommen, J., Vanden Berghe, G., Van Oudheusden, D.: A personalized tourist trip design algorithm for mobile tourist guides. Applied Artificial Intelligence **22**(10), 964–985 (2008). DOI 10.1080/08839510802379626
23. Weng, D., Chen, R., Zhang, J., Bao, J., Zheng, Y., Wu, Y.: Pareto-optimal transit route planning with multi-objective monte-carlo tree search. IEEE Transactions on Intelligent Transportation Systems **22**(2), 1185–1195 (2021). DOI 10.1109/tits.2020.2964012
24. Wessel, N., Allen, J., Farber, S.: Constructing a routable retrospective transit timetable from a real-time vehicle location feed and gtfs. Journal of Transport Geography **62**, 92–97 (2017). DOI 10.1016/j.jtrangeo.2017.04.012
25. Wong, J.: Leveraging the general transit feed specification for efficient transit analysis. Transportation Research Record: Journal of the Transportation Research Board **2338**(1), 11–19 (2013). DOI 10.3141/2338-02
26. Xia, D., Ma, J., Sharif Azadeh, S.: Integrated timetabling and vehicle scheduling of an intermodal urban transit network: A distributionally robust optimization approach. Transportation Research Part C: Emerging Technologies **162**, 104610 (2024). DOI 10.1016/j.trc.2024.104610
27. Zhang, Y., Cats, O., Azadeh, S.S.: Dynamic preference-based multi-modal trip planning of public transport and shared mobility. Transportation Research Part E: Logistics and Transportation Review **202**, 104286 (2025). DOI 10.1016/j.tre.2025.104286
28. Zhou, M., Wang, Y.: Optimal routing and request selection for multiple service routes in a demand-adaptive transit system. Frontiers of Engineering Management **12**(4), 983–1004 (2025). DOI 10.1007/s42524-025-4174-3